ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PRACTICE EXAM 4 · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 4 Technical Programming practice exam 4.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Accept any correct code or design that does the same job, including other variable names, other correct Python methods and other flowchart layouts that use the correct symbols. Follow through (consistent accuracy) where an earlier error is carried forward.

SECTION A

QUESTION 1

1.1 1.1.1 B ✓ 1.1.2 D ✓ 1.1.3 A ✓ 1.1.4 C ✓ 1.1.5 B ✓ 1.1.6 A ✓ 1.1.7 C ✓ 1.1.8 D ✓ 1.1.9 B ✓ 1.1.10 D ✓ (10)

1.2 1.2.1 True ✓ 1.2.2 False ✓ 1.2.3 True ✓ 1.2.4 False ✓ 1.2.5 True ✓ 1.2.6 True ✓ 1.2.7 False ✓ 1.2.8 True ✓ 1.2.9 False ✓ 1.2.10 True ✓ (10)

1.3 1.3.1 C ✓ 1.3.2 G ✓ 1.3.3 F ✓ 1.3.4 J ✓ 1.3.5 E ✓ 1.3.6 A ✓ 1.3.7 I ✓ 1.3.8 B ✓ 1.3.9 H ✓ 1.3.10 D ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1 Start terminator ✓; display menu and input choice (parallelogram) ✓; decision diamonds for choices 1, 2 and 3 with the matching process boxes ✓✓; decision for choice 4 leading to Stop ✓; loop arrow back to the menu after each choice / invalid choice ✓ (6)

2.2 Empty list ✓; while loop ✓; menu printed and choice input ✓; option 1 converts and appends ✓; option 2 loops and prints ✓; option 3 average, guarded against an empty list ✓; option 4 breaks / stops the loop ✓; else prints "Invalid choice" ✓: (8)

```
readings = []
while True:
    print("1 Add a reading")
    print("2 Show all readings")
    print("3 Show the average")
    print("4 Quit")
    choice = input("Choice: ")
    if choice == "1":
        temp = float(input("Temperature: "))
        readings.append(temp)
    elif choice == "2":
        for r in readings:
            print(r)
    elif choice == "3":
        if len(readings) > 0:
            print("Average:", sum(readings) / len(readings))
        else:
            print("No readings yet")
    elif choice == "4":
```

```
        break
    else:
        print("Invalid choice")
```

2.3 The user may type text or an impossible value, which would crash the program or store wrong data ✓. Use try/except around the conversion and check that the value is in a sensible range (e.g. −20 to 150 °C) before adding it ✓. (2)

[16]

QUESTION 3

3.1.1 Robot box with name, _battery and status() ✓; Arm box with joints and a method, e.g. status(), rotate() ✓✓; AGV box with an attribute, e.g. speed / location, and a method, e.g. drive(), status() ✓✓; inheritance arrows (hollow triangle) pointing from the children to Robot ✓ (6)

3.1.2 class Arm(Robot) ✓; __init__ with joints ✓; super().__init__(…) ✓; self.joints stored ✓; status() overridden ✓, returning the parent status plus the joints ✓: (6)

```
class Arm(Robot):
    def __init__(self, name, joints, battery=100):
        super().__init__(name, battery)
        self.joints = joints

    def status(self):
        return super().status() + ", " + str(self.joints) + " joints"
```

3.2 Polymorphism: the same method call (status()) works on objects of different classes, and each object uses its own version of the method ✓✓. Loop ✓✓: (4)

```
for robot in fleet:
    print(robot.status())
```

3.3 Encapsulation keeps an object's data inside the class and lets other code use it only through the class's methods, which protects it from wrong changes ✓✓. Method ✓✓: (4)

```
    def get_battery(self):
        return self._battery
```

[20]

QUESTION 4

4.1 Function definition ✓; open in write mode ✓; loop over the readings ✓; build the line with commas ✓; newline at the end of each line ✓: (5)

```
def save_log(readings, filename):
    with open(filename, "w") as f:
        for time, robot, temp in readings:
            f.write(time + "," + robot + "," + str(temp) + "\n")
```

4.2 try ✓; open and loop over the lines ✓; strip and split on commas ✓; temperature converted to int ✓; tuple appended and list returned ✓; except FileNotFoundError returns [] ✓: (6)

```
def load_log(filename):
    readings = []
    try:
        with open(filename) as f:
            for line in f:
                time, robot, temp = line.strip().split(",")
                readings.append((time, robot, int(temp)))
    except FileNotFoundError:
        return []
    return readings
```

4.3 3 warnings ✓✓✓ (63, 81 and 60 are all 60 or more; award 1 mark only for "2 warnings" from missing the boundary value 60) (3)

4.4 Any TWO ✓✓: data in a list is lost when the program stops or the power fails; a file keeps a permanent record for reports and fault-finding; the file can be opened in a spreadsheet or by other programs; the data can be loaded again the next time the program runs. (2)

[16]

QUESTION 5

5.1 Each error identified ✓ and corrected ✓: line 1 – missing colon: def average(values): ✓✓; line 2 – = must be == : if len(values) == 0: ✓✓; line 7 – value is not defined: return total / len(values) ✓✓; last line – a string cannot be joined to a float: print("Average: " + str(average(temps))) or print("Average:", average(temps)) ✓✓ (8)

5.2 Six sensible values with the correct expected results ✓✓✓✓✓, covering normal, boundary and invalid data, e.g.: (6)

Test value	Type of data	Expected result
35	normal	OK
59	boundary	OK
60	boundary	WARN
79	boundary	WARN
80	boundary	STOP
"hot" or −500	invalid	error message, no crash

5.3 Line 1 – missing semicolon ✓: int ledPin = 13; ✓. Line 9 – C is case sensitive, digitalwrite is wrong ✓: digitalWrite(ledPin, LOW); ✓ (4)

[18]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE